VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Experiment No. 00

Aim: - Introduction to Java Programming Lab.

Software Requirement: - Java Development Kit (JDK), Integrated Development Environment (IDE) (NetBeans), Java Runtime Environment (JRE).

Theory: -

Introduction to Java

Java is a class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is intended to let application developers Write Once and Run Anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. Java was developed by James Gosling at Sun Microsystems Inc. in May 1995 and later acquired by Oracle Corporation and is widely used for developing applications for desktop, web, and mobile devices.

Java is known for its simplicity, robustness, and security features, making it a popular choice for enterprise-level applications. Java applications are compiled to byte code that can run on any Java Virtual Machine. The syntax of Java is similar to C/C++.

Java makes writing, compiling, and debugging programming easy. It helps to create reusable code and modular programs.

History of Java

Java's history is as interesting as it is impactful. The journey of this powerful programming language began in 1991 when James Gosling, Mike Sheridan**,** and Patrick Naughton, a team of engineers at Sun Microsystems known as the "Green Team," set out to create a new language initially called "Oak." Oak was later renamed Java**,** inspired by Java coffee, and was first publicly released in 1996 as Java 1.0. This initial version provided a no-cost runtime environment across popular platforms, making it accessible to a broad audience. Arthur Van Hoff rewrote the Java 1.0 compiler to strictly comply with its specifications, ensuring its reliability and cross-platform capabilities.

- Java evolved over time, with Java 2 introducing multiple configurations tailored for different platforms, showcasing its versatility.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

- In 1997, Sun Microsystems aimed to formalize Java through the ISO standards body but eventually withdrew from the process.
- Despite not formalizing through ISO, Sun Microsystems offered most Java implementations at no cost, earning revenue by licensing specialized products such as the Java Enterprise System.
- A significant milestone in Java's history occurred on November 13, 2006, when Sun Microsystems released a large portion of the Java Virtual Machine **(**JVM**)** as free, open-source software.
- By May 8, 2007, the core JVM code was fully available under open-source distribution terms.
- Java was designed with core principles: simplicity, robustness, security, high performance, portability, multi-threading**,** and dynamic interpretation. These principles have made Java a preferred language for various applications, including mobile devices, internet programming, gaming, and e-business.
- Today, Java continues to be a cornerstone of modern software development, widely used across industries and platforms.

Key Features of Java

1. Platform Independent

Compiler converts source code to byte code and then the JVM executes the bytecode generated by the compiler. This byte code can run on any platform be it Windows, Linux, or macOS which means if we compile a program on Windows, then we can run it on Linux and vice versa. Each operating system has a different JVM, but the output produced by all the OS is the same after the execution of the byte code. That is why we call java a platform-independent language.

2. Object-Oriented Programming

Java is an object-oriented language, promoting the use of objects and classes. Organizing the program in the terms of a collection of objects is a way of object-oriented programming, each of which represents an instance of the class.

The four main concepts of Object-Oriented programming are:

- Abstraction
- Encapsulation
- Inheritance
- Polymorphism

3. Simplicity

Java's syntax is simple and easy to learn, especially for those familiar with C or C++. It eliminates complex features like pointers and multiple inheritances, making it easier to write, debug, and maintain code.

4. Robustness

Java language is robust which means reliable. It is developed in such a way that it puts a lot of effort into checking errors as early as possible, that is why the java compiler is able to detect even those errors that are not easy to detect by another programming language. The main features of java that make it robust are garbage collection, exception handling, and memory allocation.

5. Security

In java, we don't have pointers, so we cannot access out-of-bound arrays i.e it shows ArrayIndexOutOfBound Exception if we try to do so. That's why several security flaws like stack corruption or buffer overflow are impossible to exploit in Java. Also, java programs run in an environment that is independent of the os(operating system) environment which makes java programs more secure.

6. Distributed

We can create distributed applications using the java programming language. Remote Method Invocation and Enterprise Java Beans are used for creating distributed applications in java. The java programs can be easily distributed on one or more systems that are connected to each other through an internet connection.

7. Multithreading

Java supports multithreading, enabling the concurrent execution of multiple parts of a program. This feature is particularly useful for applications that require high performance, such as games and real-time simulations.

8. Portability

As we know, java code written on one machine can be run on another machine. The platform-independent feature of java in which its platform-independent bytecode can be taken to any platform for execution makes java portable. WORA(Write Once Run Anywhere) makes java application to generates a '.class' file that corresponds to our applications(program) but contains code in binary format. It provides ease t architecture-neutral ease as bytecode is not dependent on any machine architecture. It is the primary reason java is used in the enterprising IT industry globally worldwide.

9. High Performance

Java architecture is defined in such a way that it reduces overhead during the runtime and at sometimes java uses Just In Time (JIT) compiler where the compiler compiles code on-demand basis where it only compiles those methods that are called making applications to execute faster.

How Java Code Executes?

1. Creating the Program

Java programs are written using a text editor or an Integrated Development Environment (IDE) like IntelliJ IDEA**,** Eclipse, or NetBeans. The source code is saved with a .java extension**.**

2. Compiling the Program

The Java compiler (javac) converts the source code into bytecode, which is stored in a .class file. This bytecode is platform-independent and can be executed on any machine with a JVM**.**

3. Running the Program

The JVM executes the compiled bytecode, translating it into machine code specific to the operating system and hardware.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Essential Java Terminologies

1. Java Virtual Machine(JVM)

The **JVM** is an integral part of the **Java platform**, responsible for executing Java bytecode. It ensures that the output of Java programs is consistent across different platforms.

- Writing a program is done by a java programmer like you and me.
- The compilation is done by the **JAVAC** compiler which is a primary Java compiler included in the Java development kit (JDK). It takes the Java program as input and generates bytecode as output.
- In the Running phase of a program, **JVM** executes the bytecode generated by the compiler.

Now, we understood that the function of Java Virtual Machine is to execute the bytecode produced by the compiler. Every **Operating System** has a different JVM but the output they produce after the execution of bytecode is the same across all the operating systems. This is why Java is known as a **platform-independent language.**

2. Bytecode

Bytecode is the intermediate representation of Java code, generated by the Java compiler. It is platform-independent and can be executed by the JVM.

3. Java Development Kit(JDK)

While we were using the term **JDK** when we learn about bytecode and JVM. So, as the name suggests, it is a complete Java development kit that includes everything including **compiler, Java Runtime Environment (JRE)**, **Java Debuggers**, **Java Docs**, etc. For the program to execute in java, we need to install JDK on our computer in order to create, compile and run the java program.

4. Java Runtime Environment (JRE)

JDK includes JRE. **JRE** installation on our computers allows the java program to run, however, we cannot compile it. JRE includes a browser, JVM, applet support, and plugins. For running the java program, a computer needs JRE.

5. Garbage Collector

In Java, programmers can't delete the objects. To delete or recollect that memory JVM has a program called **Garbage Collector**. Garbage Collectors can recollect the objects that are not referenced. So **Java** makes the life of a programmer easy by **handling memory management**. However, **programmers should be careful about their code whether they are using objects that have been used for a long time**. Because Garbage cannot recover the memory of objects being referenced.

6. ClassPath

The **Classpath** is the file path where the java runtime and Java compiler look for **.class** files to load. By default, JDK provides many libraries. If you want to include external libraries, they should be added to the classpath.

Basically everything in java is represented in Class as an object including the main function.

Advantages of Java

- Platform independent: Java code can run on any platform that has a Java Virtual Machine (JVM) installed, which means that applications can be written once and run on any device.
- Object-Oriented: Java is an object-oriented programming language, which means that it follows the principles of encapsulation, inheritance, and polymorphism.
- Security: Java has built-in security features that make it a secure platform for developing applications, such as automatic memory management and type checking.
- Large community: Java has a large and active community of developers, which means that there is a lot of support available for learning and using the language.
- Enterprise-level applications: Java is widely used for developing enterprise-level applications, such as web applications, e-commerce systems, and database systems.

VISION	MISSION
To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.	1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities. 2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program. 3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

Disadvantages of Java

1. Performance: Java can be slower compared to other programming languages, such as C++, due to its use of a virtual machine and automatic memory management.

2. Memory management: Java's automatic memory management can lead to slower performance and increased memory usage, which can be a drawback for some applications.

Conclusion

Hence, thus I have studied the Java Programming Lab Successfully.